

Sede	Sede Andina
Localidad	Bariloche
Escuela	Escuela de Producción y Tecnología
Carrera	Ingeniería en Computación

Programa analítico de	Programación II	Código Guaraní
	Plan de estudio: ICOMP 2021-V9	B6008
	Ubicación en el plan de estudios	2 año - 1º CUATRIMESTRE

Correlativas vigentes	Para cursar	Cursada aprobada	Programación I (B6003)
		Materia aprobada	Sin correlativas vigentes
	Para rendir	Materia aprobada	Programación I (B6003)
Ciclo lectivo: 2023	Régimen de cursada	CUATRIMESTRAL	
	Período de dictado	1º CUATRIMESTRE	
Carga horaria semanal: 6		Carga horaria total: 96	
Horas teóricas totales: 48		Horas prácticas totales: 48	
Carga horaria virtual: 0		Horas de estudio extra clase recomendadas: 20	

Profesor/a	MARTIN RENE VILUGRON
Equipo de docencia	Sin equipo de docencia

Fundamentación

La programación es uno de los pilares fundamentales de la carrera, dado que forma la base sobre la cual se construyen todas las demás habilidades técnicas y analíticas de un profesional en el área de la informática. Por este motivo, el objetivo principal de esta asignatura es que los estudiantes aprendan a programar de forma correcta y que sean capaces de desarrollar software de alta calidad y de manera eficiente desde los inicios de su acercamiento a la programación como disciplina.

Esta cátedra adopta la estrategia "Late-Objects" formulada por Cay Horstmann, para simplificar la transición desde C al introducir el grueso de los nuevos conceptos, luego de tratar la mayor cantidad de similitudes con el lenguaje de programación C, visto en la cátedra correlativa.

La cátedra hace un uso intensivo de herramientas de análisis y verificación automática, no solo por la realimentación a los estudiantes y la velocidad de la misma, sino por el uso que estas herramientas tienen en su futura carrera profesional.

Se les otorgarán las herramientas para que los estudiantes puedan diseñar una solución y luego implementarla a partir de una mirada más amplia del concepto de artefacto de software.

Propósitos de la asignatura

Comprensión Profunda del Lenguaje Java: Java es un lenguaje de programación ampliamente utilizado en la industria por su robustez, portabilidad y seguridad. Los estudiantes aprenderán a manejar este lenguaje de manera avanzada, comprendiendo sus características y aplicando las mejores prácticas en su uso.

Diseño Orientado a Objetos: Se enfatizará el uso de la programación orientada a objetos (POO) para estructurar el software de manera modular y reutilizable. Los estudiantes aprenderán a diseñar y desarrollar sistemas complejos utilizando principios sólidos de POO.

Resolución de Problemas: Se desarrollarán habilidades para descomponer problemas complejos en partes manejables y diseñar soluciones eficientes. Se fomentará un pensamiento crítico y una metodología sistemática para abordar y resolver problemas.

Calidad y Mantenibilidad del Código: Se destacará la importancia de escribir código limpio, documentado y fácilmente mantenible. Los estudiantes aprenderán sobre pruebas unitarias, refactorización y otras prácticas que mejoran la calidad del software.

Trabajo en Equipo y Herramientas de Desarrollo: Se promoverá el trabajo colaborativo y el uso de herramientas modernas de desarrollo, como sistemas de control de versiones y entornos de desarrollo integrados (IDEs).

Aplicaciones de estructuras de datos, algoritmos y análisis de complejidad, vistos en parte en Programación 1 y profundizados aquí con usos prácticos.

Contenidos mínimos según plan de estudio

Entrada/salida de información por archivos. Archivos de acceso aleatorio. Memoria dinámica; reserva, liberación de memoria y ciclo de vida de objetos. Puntero a función. Patrones de diseño. Refactorización y Deuda Técnica. Estructuras de datos. Tipos de datos definidos por el usuario (TAD: Tipo Abstracto de Dato). Listas, pilas, colas, tablas de hash, árboles, colas priorizadas, conjuntos y grafos.

Propuesta metodológica

La asignatura constará de clases teóricas y clases prácticas diferenciadas, adoptando una estrategia práctica para maximizar el aprendizaje y la aplicación de los conceptos enseñados.

En las clases teóricas se desarrollarán los temas del programa de la asignatura, incluyendo múltiples ejemplos que faciliten la asimilación de los contenidos conceptuales. Se utilizarán ejemplos prácticos para relacionar cada uno de los temas vistos, fomentando la interacción del alumno con el objetivo de que logre una actitud activa para, a través de su propio razonamiento, crear soluciones creativas a problemas planteados durante las clases.

En las clases prácticas se plantearán distintos conjuntos de ejercicios para favorecer la asimilación de los conceptos vistos en la materia. Al inicio de cada sesión práctica, el profesor hará una breve explicación con ejemplos similares a los ejercicios propuestos en la práctica. Los estudiantes trabajarán en la resolución de estos ejercicios, aplicando los conocimientos teóricos en un entorno práctico.

Se fomentará el uso de herramientas modernas de desarrollo de software para mejorar la experiencia de aprendizaje y preparar a los estudiantes para el entorno profesional. Las principales herramientas y metodologías a utilizar son:

- git para el control de cambios, permitiéndoles gestionar sus proyectos de manera eficiente y colaborar con sus compañeros.
- Github será utilizado como plataforma para alojar y compartir repositorios, facilitando el seguimiento del progreso de los estudiantes.
- La combinación de las herramientas de verificación PMD, CheckStyle, SpotBugs y JaCOCO con reglas personalizadas y adaptadas a la cátedra.
- Gradle Los estudiantes aprenderán a usar esta herramienta de construcción de software para gestionar las dependencias del proyecto, compilar el código y ejecutar las pruebas. Gradle simplifica el proceso de construcción y permite integrar diversas herramientas en el flujo de desarrollo.
- El entorno de desarrollo IntelliJ Community Edition, el cual es una herramienta de desarrollo avanzada que integra todas las herramientas descritas anteriormente.

Se fomentará el aprendizaje a través de la resolución de problemas prácticos, estimulando a que los estudiantes propongan y programen soluciones creativas a distintos problemas propuestos. Este enfoque práctico permitirá a los estudiantes aplicar los conocimientos teóricos en situaciones reales, desarrollando habilidades críticas para su futuro profesional.

Todo el contenido de la cátedra quedará alojado en el repositorio <https://github.com/INGCOM-UNRN-PII/cursada-2024>, el mismo irá siendo completado a lo largo del cuatrimestre.

Ajustes para estudiantes con discapacidad

La UNRN desarrolla políticas en torno a accesibilidad académica para estudiantes con discapacidad y acompañamiento de equipos docentes, en consonancia con las normativas nacionales y orientaciones del CIN. Al inicio de la cursada se espera contar con el relevamiento del espacio de APD (Asistencia Pedagógica en discapacidad) sobre los/las estudiantes que cursen la materia con el fin de identificar barreras y definir las configuraciones de apoyo necesarias.

Unidades

Nombre de la unidad: Java desde C

Semanas: 1,2

Contenidos: El objetivo de esta unidad es proporcionar a los estudiantes que ya tienen conocimientos en programación en C una transición suave y comprensiva hacia la programación en Java. Se abordarán las similitudes y diferencias entre ambos lenguajes, facilitando la adaptación a Java mediante ejemplos y ejercicios prácticos.

Introducción a Java; Historia y evolución de Java. Características principales de Java. Comparación entre C y Java. Sintaxis y Estructura del Lenguaje
Estructura de un programa en Java; Diferencias sintácticas entre C y Java. Tipos de datos y variables en Java. Operadores y expresiones en Java.
Control de Flujo; Instrucciones de control de flujo (if, else, switch). Bucles (for, while, do-while). Comparación de estructuras de control entre C y Java.
Funciones; Declaración y definición de funciones (métodos estáticos) en Java. Parámetros y retorno de métodos. Sobrecarga de métodos. Diferencias entre funciones en C y métodos en Java.

Actividades practicas: TP1-2024 - Puesta en marcha

El objetivo de esta práctica es garantizar que todos estemos en la misma página con respecto a las herramientas que utilizaremos en el cuatrimestre, así como familiarizarnos con el ciclo de trabajo.

Durante la cursada, utilizaremos la versión del JDK 17, la misma es la versión de soporte extendido (LTS) vigente por el 2024. Utilizaremos IntelliJ Idea 2024 Community Edition como entorno de desarrollo integrado, y los proyectos individuales están basados en Gradle.

TP2-2024 - Java desde C

El objetivo de esta práctica es aprovechar que la sintaxis del lenguaje que utilizaremos es prácticamente la misma para familiarizarnos con las diferencias del lenguaje con respecto a C, reutilizando consignas de Programación 1.

En esta práctica, no está permitido el uso de la librería estándar de Java, más allá del [java.util.Scanner]

Bibliografía obligatoria: Schildt, H. (2022). Java: A beginner's guide (Ninth edition). McGraw Hill.

Bibliografía complementaria: Lopez Roman, L. (2011). Programacion estructurada y orientada a objetos: Un enfoque algoritmico. Alfaomega Grupo Editor.

Nombre de la unidad: Excepciones, arreglos y archivos

Semanas: 3,4

Contenidos: El objetivo de esta unidad es comprender el manejo de excepciones en Java, situándolas en el contexto de la transición entre la programación estructurada clásica y la orientación a objetos. Además, se abordarán los arreglos (arrays) y el manejo de archivos, mostrando cómo estos temas aplican las dos principales familias de excepciones en Java.

Manejo de Excepciones; Introducción a las excepciones en Java. Bloques try-catch-finally. Diferencias en el manejo de errores entre C y Java.

Entrada/Salida (I/O); Introducción a las clases de I/O en Java (java.io y java.nio). Lectura y escritura de archivos en Java. Comparación entre funciones de I/O en C y métodos de I/O en Java.

Actividades practicas: TP3-2024 - Arreglos y excepciones: El objetivo de esta práctica es familiarizarnos con los arreglos en Java, viendo las diferencias con los de C y también tener el primer contacto con Excepciones, siendo que las operaciones con arreglos naturalmente lanzan excepciones sin tipo.

TP4-2024 - Archivos y excepciones: El objetivo de esta practica es comenzar a utilizar más clases y trabajar con archivos empleando la API java.nio y seguir trabajando con excepciones, ya que los mismos lanzan excepciones con tipo.

Bibliografía obligatoria: Schildt, H. (2022). Java: A beginner's guide (Ninth edition). McGraw Hill.

Bibliografía complementaria: Lopez Roman, L. (2011). Programacion estructurada y orientada a objetos: Un enfoque algoritmico. Alfaomega Grupo Editor.

Nombre de la unidad: Orientación a Objetos

Semanas: 5,6,7,8

Contenidos: El objetivo de esta unidad es proporcionar a los estudiantes una comprensión sólida de los conceptos fundamentales de la programación orientada a objetos (OOP), el análisis y diseño orientado a objetos, y su aplicación en Java. Se explorarán temas clave como la definición y distinción entre clase y objeto, asociaciones, ciclo de vida de un objeto, encapsulamiento, colaboración entre objetos, herencia, identidad de objetos, el protocolo equals/hashcode y los diferentes tipos de polimorfismo. Asimismo, se verá el tema 'generics' para su posterior uso en la unidad de estructuras de datos.

Actividades practicas: TP5-2024 - Análisis Orientado a Objetos I; El objetivo de esta práctica es ir introduciéndonos en los conceptos de clases y objetos a través del análisis de diferentes contextos, pero también para ir viendo como otras personas ven la misma cosa, al requerir de la revisión entre pares.

TP6-2024 - Arreglos III; El objetivo de esta practica es tomar las funciones de los prácticos 3 y 4 en clases que encapsulen un arreglo en uno propio, incluyendo funcionalidad para guardar en archivos a partir del mismo.

TP7-2024 - Análisis Orientado a Objetos II; En este trabajo, le daremos una nueva vuelta de tuerca a los contextos analizados en el TP5, incorporando lo visto de Orientación a Objetos. Esta práctica también se desarrolló con revisión de pares en el Campus.

TP8-2024 - Calculadora; El objetivo de esta practica es implementar una calculadora orientada a objetos aplicando polimorfismo, una de las técnicas más importantes del paradigma.

TP9-2024 - Arreglos Genéricos; El objetivo de esta practica, es continuar desarrollando el arreglo desarrollado en el TP6, haciendo que el mismo emplee Genéricos, para que pueda almacenar cualquier tipo de referencia.

Bibliografía obligatoria: Schildt, H. (2022). Java: A beginner's guide (Ninth edition). McGraw Hill.

Bibliografía complementaria: Lopez Roman, L. (2011). Programacion estructurada y orientada a objetos: Un enfoque algoritmico. Alfaomega Grupo Editor.

Nombre de la unidad: Patrones de diseño

Semanas: 9,10,11

Contenidos: El objetivo de esta unidad es introducir a los estudiantes a los patrones de diseño orientado a objetos, mostrando cómo estos patrones pueden resolver problemas comunes en el desarrollo de software. Los estudiantes aprenderán a reconocer, aplicar y adaptar estos patrones en sus proyectos de Java, mejorando la calidad y mantenibilidad de su código. Introducción a los Patrones de Diseño; Definición y origen de los patrones de diseño. Importancia de los patrones de diseño en el desarrollo de software. Clasificación de los patrones de diseño: creacionales, estructurales y de comportamiento.

Actividades practicas: TP10-2024 - Patrones de Diseño; En esta practica, extenderemos el arreglo del TP anterior, para que el mismo emplee patrones de diseño.

Bibliografía obligatoria: Gamma, E. (Ed.). (1995). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Bibliografía complementaria: Sin bibliografía complementaria

Nombre de la unidad: Principios SOLID y Refactorización

Semanas: 12

Contenidos: El objetivo de esta unidad es proporcionar a los estudiantes una comprensión profunda de los principios SOLID, que son fundamentales para el diseño orientado a objetos de software, así como técnicas de refactorización para mejorar el código existente. También se introducirá el concepto de deuda técnica y cómo gestionarla en proyectos de software.

Actividades practicas: TP11-2024 - SOLIDificación de las prácticas; En esta practica analizaremos el código que hemos desarrollado antes, indicando si cumple o no con los criterios SOLID, así como el planteo de las refactorizaciones para que lo haga.

Bibliografía obligatoria: Martin, R. C. (Ed.). (2009). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

Bibliografía complementaria: Sin bibliografía complementaria

Nombre de la unidad: Estructuras de datos

Semanas: 13,14,15,16

Contenidos: El objetivo de esta unidad es proporcionar a los estudiantes una comprensión completa de las estructuras de datos fundamentales y los tipos abstractos de datos (TAD). Los estudiantes aprenderán a implementar y utilizar diversas estructuras de datos en Java, comprendiendo sus características, ventajas y desventajas.

Actividades practicas: TP12-2024 - Estructuras basadas en Nodos; En esta practica, implementaremos estructuras de datos basadas en Nodos y les daremos uso en aplicaciones simples.

TP13-2024 - Árboles; Aqui implementaremos un arbol binario

Bibliografía obligatoria: Introduction to Java Programming and Data Structures - 11ava edición, Y. Daniel Liang - Pearson

Bibliografía complementaria: Sin bibliografía complementaria

Propuesta de evaluación

La evaluación del curso consiste en tres exámenes parciales con una única instancia de recuperación más la entrega de Trabajos Prácticos.

Para la regularización de la materia es necesario entregar todas las practicas, la revisión de las mismas, no tiene previsto el uso de calificación; solo una indicación de que cumple los objetivos (OK), o no; (Corregir).

El tercer examen puede ser reemplazado por la elaboración de un Trabajo Integrador.

Asignatura posible de ser promocionada sin examen final

Condiciones de regularidad y régimen de promoción

Se establece que la materia es promocionable con la escala Promoción 7 - Promoción.

El régimen resuelve las siguientes condiciones: El alumno alcanzará esta condición si obtiene en los parciales una nota mayor o igual a siete (7) en cada uno y el 100% de entrega de Trabajos Prácticos, en donde se admitirá como máximo 1(uno) a revisar. La nota final será el promedio entre las evaluaciones parciales.

Asignatura posible de ser rendida libre
--

Se establece el régimen para las/los estudiantes que deseen rendir libre de las siguiente manera: Un repositorio en GitHub conteniendo un proyecto desarrollado aplicando las tecnicas y herramientas descriptas en el presente programa, sobre este código se haran preguntas conceptuales y solicitudes de cambio durante el examen.
--

Requisitos de acreditación

El alumno será regularizado en la materia si: obtiene en cada parcial, o recuperatorio en el caso de que corresponda, una calificación mayor o igual a cuatro (4) y el 100% de los Trabajos Prácticos entregados.

Fechas tentativas de evaluación
--

Se prevé aproximadamente, la entrega de un Trabajo Practico por semana. El primer parcial se ubicará al final del primer mes de cursada. El segundo parcial se ubicará al final del segundo mes de cursada. El tercer parcial se ubicará al final del cuarto mes, pero no menos de una semana antes de los recuperatorios. Y en la ultima clase del cuatrimestre se ubicará el recuperatorio de los exámenes anteriores.
--